

Fidèle au matériel du cours (cours 02 – 10, 12 ; **cours 11 Scrollytelling non évalué**). À copier dans Figma. Tout est

dense exprès : tu tailleras à la mise en page.

RECTO — LE CODE D3 QUI TRAITE DE TOUT

★ MÉGA-EXEMPLE : bar chart D3 complet, intégralement commenté

Cet exemple couvre **à lui seul** : convention des marges, `scaleBand`, `scaleLinear`, axes, data binding `.data().join()`, barres qui partent du bas, labels, tooltip, transitions et événements. C'est LE pattern récurrent de l'examen.

```
import { select } from 'd3-selection' // sélection / manipulation DOM
import { scaleBand, scaleLinear } from 'd3-scale'
import { axisBottom, axisLeft } from 'd3-axis'
import { max } from 'd3-array'
import { transition } from 'd3-transition'

// — 1. DONNÉES —
// (après un d3.csv, on convertit les chaînes en nombres avec +)
const data = [
  { categorie: 'Vélo',      accidents: 142 },
  { categorie: 'Piéton',   accidents: 89  },
  { categorie: 'Moto',     accidents: 283 },
  { categorie: 'Voiture',  accidents: 310 },
  { categorie: 'Trottinette', accidents: 67 },
]

// — 2. CONVENTION DES MARGES —
// On réserve de l'espace pour les axes. La zone de dessin "propre"
// fait innerW × innerH ; le <g> est décalé de (left, top).
const margin = { top: 20, right: 20, bottom: 50, left: 60 } // bottom/left + grands = place pour les axes
const width = 500, height = 350
const innerW = width - margin.left - margin.right
const innerH = height - margin.top - margin.bottom

const svg = select('#chart')
  .append('svg')
  .attr('width', width)
  .attr('height', height)

const g = svg.append('g') // groupe = zone de dessin
  .attr('transform', `translate(${margin.left}, ${margin.top})`)

// — 3. ÉCHELLES (data - pixels) —
// scaleBand : catégories discrètes - bandes de largeur égale (bar chart)
const x = scaleBand()
  .domain(data.map(d => d.categorie)) // toutes les catégories
  .range([0, innerW]) // de gauche à droite
  .padding(0.25) // espace entre barres (0-1) : + grand = barres + fines

// scaleLinear : valeurs continues - position verticale
const y = scaleLinear()
  .domain([0, max(data, d => d.accidents)]) // de 0 (TOUJOURS commencer à 0 !) au max
  .range([innerH, 0]) // Δ INVERSÉ : y=0 px est EN HAUT en SVG

// — 4. AXES (dessinés dans un <g> via .call) —
g.append('g')
  .attr('transform', `translate(0, ${innerH})`) // axe X poussé tout en bas
  .call(axisBottom(x))

g.append('g')
  .call(axisLeft(y)) // axe Y reste à gauche (x=0)

// — 5. DATA BINDING + BARRES —
// .data().join('rect') : crée les rect manquants (enter), met à jour
// les existants (update), supprime les superflus (exit).
g.selectAll('rect')
  .data(data)
  .join('rect')
  .attr('x', d => x(d.categorie)) // position via l'échelle de bande
  .attr('y', d => y(d.accidents)) // haut de la barre
  .attr('width', x.bandwidth()) // largeur d'une bande (calculée par scaleBand)
  .attr('height', d => innerH - y(d.accidents)) // hauteur = bas - sommet (barre part du bas)
  .attr('fill', 'steelblue')

// — 6. LABELS (valeur au-dessus de chaque barre) —
g.selectAll('text.val')
  .data(data)
  .join('text')
  .attr('class', 'val')
  .attr('x', d => x(d.categorie) + x.bandwidth() / 2) // centré sur la barre
  .attr('y', d => y(d.accidents) - 5)
  .attr('text-anchor', 'middle')
  .text(d => d.accidents)

// — 7. INTERACTION + TRANSITION —
// function (PAS arrow) : "this" = l'élément DOM survolé.
// Avec (event, d) => {...}, this ne pointerait PAS sur le <rect>.
const tooltip = select('#tooltip') // <div> positionné en CSS absolue

g.selectAll('rect')
  .on('mouseover', function (event, d) {
    select(this) // this = le rect survolé
      .transition().duration(150) // changement ANIMÉ (sinon instantané)
      .attr('fill', 'orange')
    tooltip
      .style('opacity', 1)
      .style('left', event.pageX + 12 + 'px') // +12 : décalé à droite pour ne pas masquer le curseur
      .style('top', event.pageY - 30 + 'px')
      .html(`<strong>${d.categorie}</strong><br>${d.accidents} accidents`) // .html pour insérer
  })
  .on('mouseout', function (event, d) {
    select(this).transition().duration(150).attr('fill', 'steelblue')
    tooltip.style('opacity', 0)
  })
```

🔑 CONCEPTS CLÉS — D3 / SVG / JS

SVG (COURS 02)

- Vectoriel** (formes mathématiques, zoom sans perte, dans le DOM) vs **matriciel/Canvas** (pixels, pixelise au zoom, hors DOM).
- Repère SVG** : origine **(0,0) en haut à gauche** ; **x** → droite, **y** → bas (inverse des maths).
- Formes : `<rect x y width height>`, `<circle cx cy r>`, `<line x1 y1 x2 y2>`, `<polygon points>`, `<path d>`, `<text>`.
 - Δ `<rect x="20" y="40" w="60" h="60">` = coin **supérieur gauche** en (20,40), PAS le centre.
- Style : `fill`, `stroke`, `stroke-width`, `opacity` (globale 0-1), `fill-opacity`, `stroke-dasharray` (tirets), `rx` (coins arrondis).
- `<path d="...">` : **M** x y (move, sans tracer) · **L** x y (line) · **H** x (horizontale) · **V** y (verticale) · **Z** (ferme). Utilisé par D3 pour arcs/courbes/cartes.
- `<g>` groupe + hérite des attributs : `transform="translate(x,y) rotate(deg) scale(k)"`.

CANVAS (COURS 02)

- Impératif** : `ctx.fillRect()`, `ctx.beginPath()` / `ctx.arc()` / `ctx.fill()`, `ctx.moveTo/lineTo/stroke`.
- Pas de DOM, pas d'interactivité native, mais **performant > 10 000 éléments**. SVG sinon (dataviz, hover, accessible).

DOM & D3-SELECTION (COURS 03)

- DOM** = arbre d'objets que JS lit/modifie. SVG est dans le DOM → manipulable comme du HTML.
- D3** = *Data-Driven Documents* (Mike Bostock, 2011, maintenu par Observable). **Modulaire** : on importe `d3-selection`, `d3-scale` ...
- Équivalences : `querySelector` → `select` · `querySelectorAll` → `selectAll` · `setAttribute` → `.attr()` · `.style` → `.style()` · `.textContent` → `.text()` · `.createElementNS` + `appendChild` → `.append()` (gère le namespace SVG) · `addEventListener` → `.on()`.
- Method chaining** : chaque méthode retourne la sélection → on enchaîne.

DONNÉES (COURS 04)

- Qualitative** : nominale (sans ordre : pays), ordinale (ordre : S/M/L), binaire (oui/non).

- Quantitative** : discrète (entiers), continue (réels).
- Formats : **CSV** (virgules, en-têtes 1re ligne), **TSV** (tabulations), **JSON** (types préservés, imbrication, APIs), **XML** (verbeux).
- Chargement (`d3-fetch`) : `csv()` / `json()` / `tsv()`. Δ **CSV retourne tout en chaînes** → convertir avec `+d.x` (sinon `"100"+"200"="100200"`).
- `d3-array` : `max`, `min`, `sum`, `mean`, `extent` (→ `[min, max]`). Tous acceptent un **accesseur** : `max(data, d => d.pop)`.
- Méthodes tableau : `map` (transforme, **nouveau** tableau) · `filter` (garde si `true`) · `includes` · `reduce(acc,d)>>...init` (agrège/compte) · `forEach` (effet de bord, ne retourne rien) · `sort((a,b)=>a-b)` croissant / `b-a` décroissant (**mute** ! `[...data].sort()` pour éviter).

DATA BINDING (COURS 04)

- `selectAll().data(arr).join('tag')` : lie données ↔ DOM.
- Enter** (donnée sans élément → créer) · **Update** (donnée + élément → modifier) · **Exit** (élément sans donnée → supprimer).
- `.join(enter=>., update=>., exit=>.)` pour contrôler chaque cas.
- Dans les callbacks : **d** = donnée liée, **i** = index. Une **clé** `.data(arr, d => d.id)` garantit l'identité des éléments entre updates.

ÉCHELLES (COURS 05) — UNE ÉCHELLE = FONCTION DOMAIN → RANGE

Échelle	Entrée → sortie	Usage
<code>scaleLinear</code>	continue → continue	positions, tailles, axes numériques
<code>scaleTime</code>	dates → continue	séries temporelles
<code>scaleSqrt</code>	continue → continue (√)	aire des bulles (cf. éthique : rayon ≠ aire)
<code>scaleQuantize</code>	continue → discrète (range régulier)	choroplèthe
<code>scaleThreshold</code>	continue → discrète (seuils définis)	classes métier
<code>scaleOrdinal</code>	catégories → valeurs	couleurs par catégorie
<code>scaleBand</code>	catégories → bandes égales	bar charts (<code>.bandwidth()</code> , <code>.padding()</code>)

- `range([innerH, 0])` pour Y : **inversé** car y=0 px est en haut.

